Lab 5 Parametryzacja, numer konta i przelew ekspresowy

1 Cel zajęć

Najpierw porządkujemy testy parametryzacją, potem zmieniamy sposób adresowania kont i dokładamy przelew ekspresowy. Kolejność nie jest przypadkowa: dobrze napisane testy sprawią, że zmiana będzie tania.

- **Po zajęciach:** napiszesz jeden test dla wielu przypadków (`@pytest.mark.parametrize`, data-driven testing),
- zmienisz kod i testy transferu, mając testy jako siatkę bezpieczeństwa,
- dodasz przelew ekspresowy w TDD i sprawdzisz swoje testy z pomocą AI.

Kod, nazwy testów, commity i prompty piszemy **po angielsku**.

2 Start

> Praca z Lab 4 musi być w `main` (PR zmergowany, CI zielone, 100% coverage). Potem:

```
git switch main
git pull
python -m pytest
git switch -c lab5
```

3 Parametryzacja testów

Kilka testów, które różnią się tylko danymi (np. cztery niepoprawne PESEL-e), można zapisać jednym testem i tabelą przypadków. To podejście to **data-driven testing**: logika testu jest jedna, a przypadki są danymi. Dodanie przypadku to jeden wiersz tabeli, a nie kopiowanie funkcji, więc łatwiej zobaczyć, co jest sprawdzane, i zauważyć, czego brakuje. `id` w `pytest.param` nadaje przypadkowi czytelną nazwę w raporcie:

```
@pytest.mark.parametrize(
    "pesel",
    [
        pytest.param("9001011234", id="too-short"),
        pytest.param("", id="empty"),
    ],
)
def test_account_with_invalid_pesel(pesel):
    assert Account("Jan", "Kowalski", pesel).pesel == "Invalid"
```

Zadanie: sparametryzuj testy błędów transferu w `test_account_registry.py` (zła kwota, brak konta nadawcy lub odbiorcy). Właśnie te testy zmienisz w kroku 4, więc od razu zobaczysz zysk. Liczba przypadków ma zostać ta sama. Zob. pytest: parametrize.

> **Zrobione, gdy:** testy zielone, 100% coverage, błędy transferu opisane tabelami z czytelnymi id, osobny commit

4 Kilka kont na jednego klienta (account_number)

Dotąd rejestr trzymał konta pod PESEL-em, więc jeden PESEL oznaczał jedno konto. W prawdziwym banku klient ma kilka kont (bieżące, oszczędnościowe), a firma też. **Nowe wymaganie: na jeden PESEL**

lub NIP może przypadać dowolnie wiele kont. PESEL nie wskazuje już jednoznacznie konta, więc każde konto dostaje własny `account_number`.

- **Numer:** tekst `PL` i 26 cyfr, generowany losowo przy tworzeniu konta, inny dla każdego konta. Opcjonalny argument konstruktora `account_number=...` pozwala go podać (zastanów się, dlaczego to ułatwia testy).
- **Rejestr:** przyjmuje `Account` i `CompanyAccount` i identyfikuje konto numerem. Niepoprawny PESEL lub NIP oraz powtórzony numer konta to `ValueError`. `find_by_account_number` zwraca konto lub `None`, a `find_by_pesel` i `find_by_nip` zwracają **listę** kont (pustą, gdy brak).
- **Transfer:** bierze numery kont zamiast PESEL-i, także między kontem osobistym i firmowym oraz między kontami tej samej osoby.

1. **Red.** Zmień testy rejestru i transferu zgodnie z nowym wymaganiem i dopisz testy numeru konta. Test „powtórzony PESEL to błąd" opisuje stare wymaganie, więc zmień go zamiast usuwać. Zobacz czerwone testy.
2. **Green.** Zaimplementuj numer konta i uprość rejestr tak, żeby trzymał jeden słownik (klucz: numer konta). Osobne commity: testy i implementacja.

> **Zrobione, gdy:** testy zielone, 100% coverage, jeden PESEL lub NIP może mieć wiele kont, transfer działa po numerze konta, rejestr jest prostszy niż przed zmianą

5 Przelew ekspresowy

Dodaj do rejestru `transfer_express(sender_number, recipient_number, amount)`:

- odbiorca dostaje dokładnie kwotę przelewu,
- nadawca płaci **kwotę plus opłatę**: **1 zł** z konta osobistego, **5 zł** z firmowego,
- saldo nie może zejść poniżej zera (z opłatą): wtedy `InsufficientFundsError` i nic się nie zmienia,
- pozostałe błędy jak w zwykłym przelewie.

Napisz testy przed kodem. Typ konta nadawcy zapisz w parametrze testu, bo różni się tylko opłata.

> **Zrobione, gdy:** testy ekspresu (opłata dla obu typów kont, brak środków) zielone, 100% coverage, osobne commity: testy i implementacja

6 AI: przegląd własnych testów

Agent ma przejrzeć Twoje testy ekspresu. Nie pisze kodu, ty decydujesz, co z jego uwag jest sensowne.

1. W `claude` lub `gemini` napisz: `Review tests/unit/test_express_transfer.py. List edge cases and scenarios that are not covered. Do not write code.`
2. Przeczytaj uwagi. Dla każdej zdecyduj: dopisuję test czy odrzucam (zastanów się dlaczego).
3. Dopisz testy do uwag, które uznałeś za słuszne. Jeśli któryś test jest czerwony, popraw kod.
4. Poproś agenta o opis PR według szablonu z Lab 4, sprawdź go i utwórz PR `lab5 → main`. Nie wklejaj do PR uwag agenta z review.

> **Zrobione, gdy:** co najmniej jeden test dodany po przeglądzie AI, wszystkie zielone, 100% coverage, PR zmergowany, CI zielone

7 Zadanie domowe: parametryzacja reszty testów

Zrób to samo w `test_account.py` i `test_company_account.py`: niepoprawny PESEL, niepoprawny NIP, kod promocyjny oraz kwota zero i ujemna we wpłacie i wypłacie. Osobna gałąź, osobny commit i PR.

Zrobione, gdy:

- powtórzenia zastąpione tabelami przypadków, liczba przypadków bez zmian,
- pytest zielony, coverage 100%, PR i merge

8 Gdy coś nie działa

- `ValueError: Account number ... already exists`: w teście dwa konta dostały ten sam numer, sprawdź argument `account_number`.
- Błąd przy parametryzacji: nazwy w pierwszym argumencie `parametrize` muszą się zgadzać z argumentami funkcji testowej, a każdy przypadek musi mieć tyle wartości, ile jest nazw.
- Zadanie domowe z Lab 4 (fixtures): dokończ, jeśli jeszcze nie zrobione.

Do poczytania: pytest: parametrize · pytest.raises.